

AOS: Adaptive Domain Independent Multi AI Agent Orchestration

Hisham K H, Irfan Nazir, Harikrishnan T B, Edwin Varghese
Department of Computer Science and Engineering
Sahrdaya College of Engineering and Technology
Kerala, India

I. PROPOSED METHODOLOGY

AOS is designed as a domain-independent orchestration layer for managing multiple AI agents and heterogeneous computational resources. The proposed methodology separates high-level agent reasoning from deterministic resource management and execution control.

The system treats models, tools, APIs, databases, and simulated sensor or robotic interfaces as computational resources managed by a common kernel-level abstraction. The complete execution process consists of task decomposition, capability identification, Capability DNA generation, resource feasibility analysis, capability-aware scheduling, dynamic resource binding, execution monitoring, failure recovery, and experience-based optimization.

The overall methodology can be represented as:

$$T \rightarrow D \rightarrow DNA \rightarrow F \rightarrow R \rightarrow B \rightarrow E \rightarrow M \rightarrow O \quad (1)$$

where T represents the input task, D represents task decomposition, DNA represents Capability DNA generation, F represents feasibility filtering, R represents resource ranking, B represents resource binding, E represents execution, M represents monitoring, and O represents the result or recovery operation.

A. A. System Architecture

The AOS architecture consists of a centralized microkernel and a set of managed agents and computational resources. The major components are:

- Unified Resource Interface
- Capability Registry
- Task Manager
- Capability DNA Extractor
- Capability-Aware Scheduler
- Dynamic Router
- Execution Manager
- Monitoring Manager
- Cognitive Memory Manager
- Agent Manager
- Communication Manager
- Cognitive Failure Manager
- Learning Manager

The kernel mediates resource access and agent communication. Consequently, resource selection, execution monitoring,

and failure recovery can be performed centrally rather than through independent application-level logic.

B. B. Unified Resource Interface

AOS provides a common interface for all computational resources. A resource may be an LLM, vision model, speech model, tool, API, database, or simulated sensor/robotic endpoint.

Each resource implements two fundamental operations:

```
describe() -> CapabilityManifest  
invoke(request) -> response
```

The *CapabilityManifest* describes the resource using structured information including:

- resource identifier,
- resource class,
- supported capabilities,
- input and output schemas,
- cost model,
- latency model,
- quality priors,
- availability,
- risk class.

This unified interface allows heterogeneous resources to be managed using the same scheduling and execution mechanisms.

C. C. Capability Registry

The Capability Registry maintains the available resources and their corresponding capability manifests. Resources periodically publish their manifests, while availability is monitored using heartbeat information.

For a given capability requirement, the registry returns the set of resources that potentially satisfy the required capabilities.

The registry therefore provides the initial candidate set for the scheduler. Resource quality priors can initially be defined statically and subsequently updated using observed execution outcomes.

D. D. Task Decomposition

The Task Manager receives the user request and converts it into a structured Task Directed Acyclic Graph (DAG).

Each node represents an executable subtask and each edge represents a dependency between subtasks.

The task decomposition process uses a schema-constrained language model to generate the DAG. The generated structure is validated before execution.

The validation process checks:

- 1) DAG acyclicity,
- 2) input and output type compatibility,
- 3) dependency consistency,
- 4) capability satisfiability.

If validation fails, a bounded repair loop provides the validation error to the planner for correction. If the required capability cannot be satisfied by any registered resource, the task is rejected or degraded through the admission-control mechanism.

E. E. Capability DNA

Capability DNA is the central representation used by AOS to describe the requirements of an individual subtask.

Each Capability DNA vector contains three major segments:

- 1) **Discrete capability flags**
- 2) **Ordinal complexity scores**
- 3) **Continuous execution constraints**

The discrete capability flags specify required capabilities such as:

- vision understanding,
- speech transcription,
- deep reasoning,
- tool calling,
- database querying,
- code generation.

The ordinal segment represents requirements such as reasoning depth, planning horizon, tool complexity, memory dependence, and parallelizability.

The continuous constraints include:

- maximum cost,
- latency SLO,
- minimum quality,
- risk tolerance.

Thus, the Capability DNA represents not only what capability is required but also the execution conditions under which the capability must operate.

F. F. Deterministic Feasibility Filtering

AOS performs deterministic feasibility filtering before resource ranking. This prevents a resource from being selected solely because it has high semantic similarity to a task while lacking one or more mandatory capabilities.

For a resource r and subtask t , the feasibility function is defined as:

$$F(r, t) = \begin{cases} 1, & C_r \supseteq C_t \wedge K(r, t) = 1, \\ 0, & \text{otherwise,} \end{cases} \quad (2)$$

where C_r represents the capabilities provided by resource r , C_t represents the capabilities required by the Capability DNA,

and $K(r, t)$ represents satisfaction of the cost, latency, quality, and risk constraints.

Only resources for which $F(r, t) = 1$ are forwarded to the ranking stage.

G. G. Capability-Aware Resource Ranking

After infeasible resources are removed, AOS ranks the remaining resources using a utility-based scoring mechanism.

The initial rule-based scoring function is:

$$S(r, t) = Q(r, t) - \lambda C(r, t) - \mu L(r, t), \quad (3)$$

where:

- $Q(r, t)$ is the estimated quality,
- $C(r, t)$ is the estimated execution cost,
- $L(r, t)$ is the estimated latency,
- λ is the cost penalty coefficient,
- μ is the latency penalty coefficient.

The resource with the highest score among the feasible candidates is selected.

The architecture can subsequently replace the rule-based scorer with a contextual bandit. In this formulation, the Capability DNA acts as the context, feasible resources act as candidate arms, and the execution outcome provides the reward signal.

H. H. Dynamic Resource Binding

AOS separates capability selection from concrete resource selection. The scheduler first determines the required capability and then dynamically binds it to a suitable available resource.

This late-binding mechanism allows resources to be replaced when they become unavailable or degraded. The replacement resource is selected using the same Capability DNA and feasibility constraints as the original resource.

This approach avoids tightly coupling an agent workflow to a specific model or computational resource.

I. I. Execution and Monitoring

The Execution Manager executes the nodes of the task DAG in dependency order. Independent nodes may be dispatched in parallel.

Every execution is monitored by the kernel. Execution telemetry includes information such as:

- selected resource,
- execution latency,
- resource utilization,
- estimated cost,
- output validation,
- execution status.

OpenTelemetry spans are generated for kernel decisions and execution events. This enables detailed analysis of scheduling decisions and runtime behavior.

J. J. Admission Control

Before execution, AOS determines whether the task can satisfy its declared constraints given the current resource and scheduling state.

The admission-control mechanism considers:

- latency SLO,
- cost ceiling,
- resource availability,
- queue state,
- capability requirements.

The system can reject an infeasible task or degrade selected requirements when possible. For example, a task may relax its minimum quality requirement or permit a lower-cost resource when the original configuration is not schedulable.

K. K. Parallel Task Execution

The task DAG enables AOS to identify independent subtasks. Subtasks without unresolved dependencies can be executed concurrently.

For independent subtasks $t_1, t_2, \dots, t_n$, the execution manager can dispatch them concurrently rather than sequentially:

$$T_{\text{parallel}} = \max_{i=1, \dots, n} L_i \quad (4)$$

where L_i represents the execution latency of subtask i .

This mechanism is intended to reduce wall-clock execution time for workflows containing parallelizable branches.

L. L. Shared Cognitive Memory

AOS provides a unified Memory Manager containing four memory mechanisms.

1) 1) *Semantic Cache*: The semantic cache stores validated outputs using a combination of Capability DNA and task embeddings.

A cached result is considered reusable only when its semantic similarity exceeds a specified threshold and its Capability DNA remains compatible with the current task.

2) 2) *Episodic Memory*: Completed executions are stored as Experience Records. Each record contains the task representation, Capability DNA profile, workflow graph, resource bindings, execution metrics, and failure/recovery information.

3) 3) *Execution Memory*: Execution memory stores intermediate results and active workflow state. The state can be snapshotted to support recovery and resource hot-swapping.

4) 4) *Semantic Memory*: Long-lived information extracted from previous executions is stored as semantic knowledge that can be retrieved during future tasks.

M. M. Multi-Agent Management

AOS supports a hierarchical multi-agent architecture consisting of Supervisor, Sub-Supervisor, and Worker agents.

The Supervisor manages high-level task execution, while Sub-Supervisors and Workers handle specialized portions of the workflow.

Agents are treated primarily as decision-making entities. Computational operations are performed through registered

resources rather than being directly coupled to individual agents.

The Agent Manager is responsible for agent creation, life-cycle management, and termination.

N. N. Kernel-Mediated Communication

All inter-agent communication passes through the kernel's communication layer.

The Communication Manager provides kernel-owned communication channels for agent messages. Centralizing communication allows the kernel to observe communication activity and detect communication failures.

The architecture also enables controlled injection of delayed, dropped, or corrupted messages during evaluation.

O. O. Cognitive Fault Tolerance

AOS introduces Cognitive Fault Tolerance as a closed-loop mechanism for handling failures that occur during planning, reasoning, resource execution, communication, or memory operations.

The recovery pipeline consists of three stages:

Detection $\rightarrow$ Classification $\rightarrow$ Recovery Policy Selection. (5)

1) 1) *Failure Detection*: The detection layer combines multiple signals:

- schema validation,
- output quality evaluation,
- timeout detection,
- cost-overflow detection,
- inter-agent contradiction detection,
- memory-staleness detection.

This enables the system to identify semantic failures in addition to conventional runtime exceptions.

2) 2) *Failure Classification*: Detected failures are classified into structured categories, including:

- planning bad decomposition,
- reasoning hallucination,
- reasoning inconsistency,
- resource outage,
- resource degradation,
- corrupted tool output,
- communication loss,
- communication corruption,
- stale memory.

The classification stage provides the context required for selecting an appropriate recovery mechanism.

3) 3) *Recovery Policy Selection*: Depending on the detected failure class, AOS can select from multiple recovery strategies:

- retry with the same resource,
- retry with feedback,
- resource substitution,
- hot-swap with state handoff,
- subtask re-decomposition,
- agent reassignment,

- memory-based recovery,
- workflow regeneration.

The initial implementation uses a taxonomy-to-strategy mapping. The advanced implementation can use a contextual bandit in which the failure class, Capability DNA, and remaining budget form the context.

P. P. Resource Hot-Swapping

AOS supports resource hot-swapping when a resource fails during the execution of a subtask.

The process consists of:

- 1) detecting the resource failure,
- 2) snapshotting the relevant execution state,
- 3) identifying feasible replacement resources,
- 4) ranking the replacement resources,
- 5) transferring the saved execution state,
- 6) resuming execution.

The state handoff mechanism allows the workflow to continue without restarting the complete task from the beginning.

The hot-swap success rate is therefore used as an important reliability metric.

Q. Q. Experience-Driven Workflow Optimization

The Learning Manager stores completed workflow executions as Experience Records.

Each record contains:

- task embedding,
- Capability DNA profile,
- workflow graph,
- resource bindings,
- execution cost,
- execution latency,
- task outcome,
- failures and recovery actions.

For a new task, similar experiences are retrieved using task embeddings and Capability DNA profiles.

When a matching workflow fragment is identified, AOS attempts subgraph transplantation. A previously validated workflow subgraph, including its structure and resource bindings, can be adapted to the new task.

This enables experience reuse at the workflow level rather than only at the response level.

R. R. Resource Quality Learning

The Learning Manager updates the quality estimates of resources based on observed execution outcomes.

An exponential moving average can be used:

$$Q_{t+1} = \alpha q_t + (1 - \alpha)Q_t, \quad (6)$$

where q_t represents the observed quality of a resource at time t , Q_t is the previous quality estimate, and α is the learning coefficient.

The updated quality estimate can subsequently influence future resource-selection decisions.

S. S. Experimental Methodology

The AOS evaluation follows an incremental methodology consisting of a baseline system, capability-aware orchestration, and advanced fault-tolerant learning.

1) 1) *Baseline Evaluation*: The baseline system uses sequential DAG execution with greedy exact capability matching. It acts as the control condition for subsequent experiments.

2) 2) *Capability-Aware Evaluation*: The capability-aware scheduler is evaluated against alternative resource-selection strategies, including:

- always-strongest resource,
- always-cheapest resource,
- random feasible resource,
- baseline greedy matching.

The evaluation measures quality, cost, latency, and scheduling overhead.

3) 3) *Fault-Injection Evaluation*: Controlled faults are introduced to evaluate the Cognitive Fault Tolerance mechanism.

The fault classes include:

- resource outage,
- silent model degradation,
- corrupted tool output,
- delayed messages,
- dropped messages,
- mutated messages,
- poisoned memory,
- budget starvation.

Recovery performance is compared against no recovery, naive retry, reflection-style recovery, rule-based recovery, and learned recovery.

T. T. Ablation Study

Ablation experiments are performed to determine the contribution of individual AOS mechanisms.

The following configurations are evaluated:

- `--no-dna`: disables Capability DNA,
- `--no-router`: disables dynamic routing,
- `--no-memory`: disables cognitive memory,
- `--no-recovery`: disables fault recovery,
- `--no-learning`: disables experience learning.

All configurations are evaluated using the same task workloads, seeds, and replayed resource responses where applicable.

U. U. Performance Metrics

The proposed system is evaluated using the following metrics.

1) *Task Performance*:

- task completion rate,
- output quality,
- capability-selection accuracy.

2) *Efficiency*:

- execution cost,
- end-to-end latency,
- scheduler decision overhead,
- kernel mediation overhead,
- resource utilization.

3) *Fault Tolerance*:

- failure detection precision,
- failure detection recall,
- classification accuracy,
- recovery success rate,
- hot-swap handoff success rate.

4) *Experience Reuse*:

- semantic cache hit rate,
- stale-hit rate,
- workflow subgraph reuse rate,
- marginal cost reduction,
- quality difference between reused and fresh execution.

V. V. *Reproducibility*

To ensure reproducibility, experiments are executed using fixed random seeds and recorded resource responses. A record/replay mechanism allows previously observed resource responses to be reused during repeated experiments.

All major AOS modules can be independently disabled through ablation switches. Kernel decisions are instrumented using OpenTelemetry, allowing scheduling, routing, memory, and recovery decisions to be analyzed after execution.